# Extrae toda la información de este examen, y crea paso a paso detallados para completar cada uno de los detalles de cada problema. Analiza y asume el rol de un experto en redes neuronales para descifrar y extraer todos los detalles y la mejor forma de resolver cada problema.

Es muy importante que cada accion quede gráficada y tenga un indice de error minimo las redes neuronales. Analiza tambien las imagenes y trata de extraer toda la informacion posible de ellas en relacion a los problemas que illustras.

Dejame saber si algo dentro de los problemas no queda claro y dame preguntas para preguntarle a mi evaluador.

Te resumo primero el contenido del examen y luego te doy, para **cada problema**, un plan paso a paso como "experto en redes neuronales", más dudas concretas para tu profesor.

## Visión general del examen

El examen parcial tiene 4 problemas: baterías vs tiempo (series de datos), intersección de curvas (pares de funciones), choque de dos proyectiles y un gemelo digital de un sensor de presión usando una tabla de no linealidad.[1]

## Problema 1 – Redes neuronales para descarga de batería

## Información que da el enunciado

Te piden: "Desarrollar un conjunto de redes neuronales para cada voltaje de descarga en función del período de descarga. Graficar todas las redes juntas."[1]

Datos (Período de descarga en minutos $x$ vs Voltaje $y$):

- Temperatura 25°C (línea roja):
  $(0, 4.20), (2, 4.05), (4, 3.95), (6, 3.88), (8, 3.82), (10, 3.77), (12, 3.72), (14, 3.68), (16, 3.63)$
  [1]
- Temperatura 10°C (línea azul):
  $(0, 3.75), (2, 3.70), (4, 3.68), (6, 3.65), (8, 3.62), (10, 3.58), (12, 3.54), (14, 3.50), (16, 3.45)$
  [1]

- Temperatura 0°C (línea naranja):
  $(0, 3.50), (2, 3.55), (4, 3.58), (6, 3.59), (8, 3.58), (10, 3.55), (12, 3.50), (14, 3.45), (16, 3.38)$.[1]
- Temperatura –10°C (línea verde):
  $(0, 3.20), (2, 3.30), (4, 3.38), (6, 3.42), (8, 3.40), (10, 3.35), (12, 3.30), (14, 3.25), (16, 3.20)$.[1]

Por la descripción de "línea roja/azul/naranja/verde", en la gráfica original seguramente aparecen las 4 curvas de voltaje vs tiempo en el mismo plano, lo que coincide con el objetivo de "graficar todas las redes juntas".[1]

## Estrategia de experto en redes neuronales

Aquí son **4 problemas de regresión 1D independientes** (uno por temperatura): $x \to y$ con muy pocos puntos, ideal para redes neuronales pequeñas tipo MLP (Perceptrón Multicapa).

Puedes usar **una red por temperatura** o una sola red con 2 entradas $(\text{tiempo}, \text{temperatura})$. El enunciado dice "para cada voltaje de descarga", así que yo interpretaría que quieren **una red distinta por temperatura** (más simple de explicar).

## Arquitectura sugerida (por temperatura)

- Entrada: 1 neurona ($x =$ período de descarga en minutos).
- Capa oculta: 1–2 capas ocultas de 5–10 neuronas, activación ReLU o tanh.
- Salida: 1 neurona lineal ($y =$ voltaje).
- Pérdida: MSE (error cuadrático medio).
- Métrica para mínimo error: MSE y MAE, ver que sean muy pequeños respecto al rango de voltaje (por ejemplo, error medio < 0.01 V es ya bastante bueno).

## Paso a paso detallado (para cada temperatura)

1. **Construir el dataset**
   - Para cada temperatura, arma un arreglo $X = [0, 2, 4, \ldots, 20]$ y un arreglo $Y$ con los voltajes correspondientes.[1]
   - Divide en entrenamiento/validación, por ejemplo 80% de puntos para entrenar, 20% para validar (aunque son pocos datos, es más por formalidad).
2. **Normalizar datos (opcional pero recomendable)**
   - Escala el tiempo $x$ a $[0, 1]$ dividiendo por 20.
   - Escala el voltaje $y$ restando la media y dividiendo por la desviación estándar, o simplemente déjalo tal cual si mantienes una red pequeña.
3. **Definir la red neuronal**
   - Input(1) → Dense(10, ReLU) → Dense(10, ReLU) → Dense(1, lineal).
   - Inicializa pesos con un esquema estándar (Glorot/He, las librerías lo hacen por ti).

4. **Configurar entrenamiento para mínimo error**
   - Optimizador: Adam con tasa de aprendizaje pequeña (por ejemplo 0.001–0.01).
   - Función de pérdida: MSE.
   - Entrena muchas épocas (p. ej. 500–2000) pero con **early stopping** en validación para evitar overfitting excesivo en tan pocos puntos.
   - Guarda el mejor modelo según el menor MSE de validación.
5. **Graficar datos y la red (cada red individual)**
   - Genera un vector de tiempos denso, por ejemplo 0, 0.1, 0.2, …, 20.
   - Pasa esos tiempos por la red para obtener voltajes predichos.
   - En una misma gráfica:
     - Dibuja los puntos reales como marcadores (círculos).
     - Dibuja la curva de la red como una línea suave.
   - Repite para cada temperatura.
6. **Graficar todas las redes juntas (como pide el examen)**
   - En una sola figura:
     - Curva real + curva de la red para 25°C en rojo.
     - Curva real + curva de la red para 10°C en azul.
     - Curva real + curva de la red para 0°C en naranja.
     - Curva real + curva de la red para –10°C en verde.[1]
   - Incluye leyenda, título y nombres de ejes.
7. **Calcular y reportar índice de error mínimo**
   - Calcula MSE y MAE sobre el conjunto completo de puntos de cada temperatura.
   - Reporta algo como:
     - MSE entrenamiento, MSE validación.
     - Error máximo absoluto (máx $|y_{\text{real}} - y_{\text{pred}}|$).
   - Explica brevemente que el error es pequeño comparado con el rango de voltaje.
8. **Comentario sobre la forma de las curvas (imagen original)**
   - A 25°C la curva baja casi monótonamente, con caída más fuerte al final (de 4.20 a 3.40).[1]
   - A menores temperaturas el voltaje inicial es menor y la curva tiende a ser algo menos estable, con ligeros aumentos iniciales antes de decaer (por ejemplo a 0°C y –10°C sube un poco al principio y luego cae).[1]
   - Esto lo puedes mencionar para mostrar que la red captura comportamientos ligeramente no-lineales en función del tiempo.

### Dudas / cosas no 100% claras

- No especifica si quieren **una red por temperatura** o una sola red con dos entradas (tiempo y temperatura).
- No indica si el objetivo es **interpolar** solo en el rango dado o también extrapolar más allá de 20 minutos.
- No indica si puedes asumir librerías específicas (Keras, PyTorch, etc.).

**Preguntas que podrías hacer al profesor**

1. ¿Desea que construyamos **una red neuronal separada por cada temperatura** o una sola red con tiempo y temperatura como entradas?
2. ¿Debemos reportar solo la gráfica o también **métricas cuantitativas de error** (MSE, MAE, error máximo)?
3. ¿Está permitido utilizar librerías como Keras/PyTorch o debemos implementar el perceptrón de forma manual?
4. ¿Es necesario considerar extrapolación del modelo más allá de 20 minutos o basta con la interpolación en el rango dado?

## Problema 2 – Redes para curvas y puntos de intersección

### Información del enunciado

Te piden: "Desarrollar redes neuronales con aprendizaje automatizado, graficando en una imagen todas las curvas" para varios pares de funciones, y "Encontrar los puntos de intersección entre las curvas".[1]

Pares de curvas:

a) $y = x^2 - 2x + 1; y = -x^2 + 5x - 2$ [1]
b) $y = x^2 + 1; y = x + 1$ [1]
c) $y = x^2 - 3x + 1; y = x + 1$ [1]
d) $y = 2 - x^2; y = x^2$ [1]
e) $y = 2x - x^2; y = x - 2$ [1]
f) $y = x^2 - 4; y = 3x - x^2 + 1$ [1]

### Interpretación como experto en redes

Aquí tienes dos tareas:

1. **Graficar las curvas reales** (fácil usando las ecuaciones).
2. **Usar redes neuronales para aproximar cada curva** y mostrar en una misma imagen las curvas reales y las aproximadas.
3. **Encontrar los puntos de intersección** entre las curvas de cada par (puedes hacerlo analíticamente y/o numéricamente).

Lo más coherente con "aprendizaje automatizado" es:

- Para cada función, generar un conjunto de datos $(x, y)$ a partir de la ecuación exacta.
- Entrenar una red neuronal de regresión que aprenda esa relación.
- Comparar gráficamente la red contra la función real.

## Paso a paso genérico por par de curvas

Repite estos pasos para cada par (a–f), cambiando solo las funciones y el rango de $x$.

1. **Elegir un rango de $x$**
   - Escoge un intervalo donde las curvas se crucen y sean interesantes, por ejemplo $[-5, 5]$ o un rango que cubra visualmente las intersecciones.
2. **Generar datos sintéticos para entrenamiento**
   - Toma muchos puntos, por ejemplo 200–500 valores de $x$ uniformemente espaciados en el rango.
   - Para cada función $f(x)$ y $g(x)$, crea datasets separados:
     - Para la primera curva: $X_f, Y_f = f(X_f)$.
     - Para la segunda curva: $X_g, Y_g = g(X_g)$.
3. **Diseñar las redes neuronales**
   - Para cada función, una MLP simple:
     - Entrada: 1 neurona ($x$).
     - 1–2 capas ocultas de 10–20 neuronas, ReLU/tanh.
     - Salida: 1 neurona lineal ($y$).
   - Pérdida: MSE; optimizador: Adam.
4. **Entrenamiento con mínimo error**
   - Entrena con los datos $(X_f, Y_f)$ y $(X_g, Y_g)$ hasta que el MSE sea muy pequeño (puedes fijarte que el error promedio sea < 1e–3 si los valores de $y$ son de orden 1–10).
   - Usa parte de los puntos para validación y early stopping.
5. **Graficar curvas reales y redes**
   - Genera un conjunto de $x$ denso (por ejemplo 1000 puntos en el mismo rango).
   - Calcula:
     - Curvas reales: $y_f = f(x), y_g = g(x)$.
     - Curvas de red: $y_{f,\text{NN}} = \text{NN}_f(x), y_{g,\text{NN}} = \text{NN}_g(x)$.
   - En una misma imagen:
     - Dibuja $f(x)$ con línea continua y la salida de la red con línea discontinua del mismo color.
     - Igual para $g(x)$ en otro color.

- Añade título indicando el par (a, b, c, ...).

6. **Encontrar puntos de intersección exactos (analíticos)**
   - Cada par son ecuaciones de segundo grado o lineales, así que puedes resolver $f(x) = g(x)$ analíticamente.
   - Por ejemplo, en (b): $x^2 + 1 = x + 1 \Rightarrow x^2 - x = 0 \Rightarrow x(x-1) = 0$, intersecciones en $x = 0$ y $x = 1$ y luego calculas $y$.
   - Repite para cada par.
7. **Encontrar intersecciones numéricas usando la red (opcional, para lucirte)**
   - Define $h(x) = \mathrm{NN}_f(x) - \mathrm{NN}_g(x)$.
   - Usa un método numérico (búsqueda de raíces, bisección, Newton, etc.) sobre $h(x)$ para estimar raíces (donde la diferencia es 0).
   - Compara con la solución analítica y reporta el error en la posición de intersección.
8. **Índice de error en las redes**
   - Calcula MSE y MAE para cada red sobre puntos de prueba independientes.
   - Calcula también el error de las intersecciones:
     - $|x_{\text{intersección, NN}} - x_{\text{intersección, analí tico}}|$.
   - Explica que la red captura correctamente la forma de las curvas y ubica bien los puntos de intersección.

## Dudas / cosas no totalmente claras

- El enunciado no especifica **rango de** $x$ a usar para entrenar y graficar las curvas.[1]
- No dice si los puntos de intersección deben sacarse **solo por métodos analíticos** o usando también la red entrenada.[1]

### Preguntas para tu profesor

1. ¿Desea que los puntos de intersección se encuentren **analíticamente** resolviendo las ecuaciones, o que también usemos la red para estimarlos numéricamente?
2. ¿Hay un **rango recomendado de** $x$ para graficar las curvas de cada par?
3. ¿Es obligatorio que todas las curvas (reales y de red) se muestren en **una sola imagen global**, o puede ser una figura por inciso (a–f)?
4. ¿Qué métrica de error prefiere que se reporte para las redes (MSE, MAE, error máximo, etc.)?

## Problema 3 – Choque de dos proyectiles

## Información del enunciado

Tienes dos proyectiles:

- Proyectil A: velocidad inicial $v_{0A} = 50\ \mathrm{m/s}$, ángulo $30^\circ$, altura inicial $y_{0A} = 10\ \mathrm{m}$.[1]
- Proyectil B: velocidad inicial $v_{0B} = 30\sqrt{2}\ \mathrm{m/s}$, ángulo $45^\circ$, altura inicial $y_{0B} = 0$.[1]
- La distancia horizontal inicial entre A y B es 150 m.[1]
- Se pide: "Determinar el tiempo de impacto y la posición $x$ e $y$ de impacto entre ambos proyectiles."[1]

Lo más natural es resolverlo con **física clásica** (ecuaciones de movimiento parabólico). Como experto en redes neuronales, podrías además mostrar cómo un modelo de red aprendería la trayectoria, pero para el examen probablemente basta con el enfoque analítico.

## Paso a paso analítico (más seguro para el examen)

1. **Plantear sistema de coordenadas**
   - Coloca, por ejemplo, el Proyectil A en $x = 0$ y el Proyectil B en $x = 150$ o viceversa.
   - Esto **no está explícito** en el enunciado (importante para preguntar), pero típicamente uno está a la izquierda y el otro 150 m a la derecha.
2. **Descomponer las velocidades iniciales**
   - $v_{0Ax} = v_{0A}\cos 30^\circ$, $v_{0Ay} = v_{0A}\sin 30^\circ$.
   - $v_{0Bx} = v_{0B}\cos 45^\circ$, $v_{0By} = v_{0B}\sin 45^\circ$.
3. **Ecuaciones de movimiento (sin resistencia del aire)**
   - Para A:
     - $x_A(t) = x_{0A} + v_{0Ax}t$.
     - $y_A(t) = y_{0A} + v_{0Ay}t - \frac{1}{2}gt^2$.
   - Para B:
     - $x_B(t) = x_{0B} + v_{0Bx}t$.
     - $y_B(t) = y_{0B} + v_{0By}t - \frac{1}{2}gt^2$.
4. **Condición de impacto**
   - Hay impacto cuando se cumple al mismo tiempo:
     - $x_A(t) = x_B(t)$.
     - $y_A(t) = y_B(t)$.
   - En la práctica:
     - Primero igualas $x_A(t) = x_B(t)$ y resuelves para $t$.
     - Luego verificas si ese $t$ hace que $y_A(t) = y_B(t)$ y que ambas $y$ sean $\geq 0$ (están por encima del suelo).
5. **Encontrar el tiempo de impacto y posición**
   - Con el valor de $t$ que cumple las ecuaciones, calcula:

    - $x_{\text{impacto}} = x_A(t) = x_B(t)$.
    - $y_{\text{impacto}} = y_A(t) = y_B(t)$.
  - Ese es el resultado que debes reportar.

### Cómo lo podrías "vestir" con redes neuronales

Si quisieras meter redes neuronales:

1. Generas muchos ejemplos de trayectorias variando $v_0$, ángulos, alturas y distancias iniciales, calculando las trayectorias con las ecuaciones de movimiento.
2. Entrenas una red neuronal que, dado un tiempo $t$ y los parámetros iniciales, prediga $x(t), y(t)$.
3. Luego usas la red en lugar de las ecuaciones para estimar el tiempo y punto de impacto.

Pero para el examen, yo recomendaría resolver analíticamente y mencionar brevemente cómo podría hacerse con una red.

### Dudas / cosas no claras

- No queda claro **desde dónde se mide la distancia de 150 m** (entre los puntos de lanzamiento, entre las posiciones iniciales sobre el eje $x$, etc.).[1]
- No indica explícitamente **si ambos proyectiles se lanzan exactamente al mismo tiempo** (se asume que sí, pero conviene confirmarlo).[1]

**Preguntas para tu profesor**

1. ¿Podemos asumir que la distancia de 150 m es entre las posiciones de lanzamiento sobre el eje horizontal (es decir, $x_{0B} = 150\,\text{m}$ y $x_{0A} = 0\,\text{m}$)?
2. ¿Ambos proyectiles se lanzan exactamente al mismo tiempo, o hay un desfase de tiempo entre lanzamientos?
3. ¿Es obligatorio resolver el problema **analíticamente**, o también se valora si mostramos cómo una red neuronal podría aproximar la solución?

## Problema 4 – Gemelo digital de un sensor (non-linearity vs pressure)

### Información del enunciado

Se pide: "Desarrollar un Gemelo Digital de un Sensor" usando una tabla de **Non-Linearity (%) vs Presión (Bar)**, obtenida de una gráfica.[1]

Datos:

- $0.0\ \text{Bar} \to 0.00\%$[1]
- $0.1 \to 0.01\%$[1]
- $0.2 \to -0.02\%$[1]

- $0.3 \to -0.01\%$ [1]
- $0.4 \to 0.02\%$ [1]
- $0.5 \to 0.05\%$ [1]
- $0.6 \to 0.09\%$ [1]
- $0.7 \to 0.12\%$ [1]
- $0.8 \to 0.06\%$ [1]
- $0.9 \to -0.05\%$ [1]
- $1.0 \to 0.00\%$ [1]

Observaciones que da el enunciado: [1]

- Máxima no linealidad en 0.7 Bar (0.12%).
- Comportamiento no lineal simétrico con pico positivo entre 0.6–0.7 Bar y valle negativo en 0.9 Bar.
- Casi nula en los extremos (0.0 y 1.0 Bar).
- Tendencia no lineal compleja con múltiples cambios de dirección. [1]

La gráfica correspondiente seguramente muestra una curva con forma de "lomo" positivo en el centro y valores cercanos a 0 en los extremos, con un pequeño valle negativo cercano a 0.9 Bar. [1]

## Estrategia de experto en redes neuronales (gemelo digital)

Un "gemelo digital" aquí se puede interpretar como: dado un valor de presión, el modelo (red neuronal) devuelve la **no linealidad esperada del sensor**, con mínimo error.

## Arquitectura sugerida

- Entrada: 1 neurona (Presión en Bar).
- Capa oculta: 1–2 capas de 10–20 neuronas, activación tanh o ReLU.
- Salida: 1 neurona lineal (Non-Linearity %).
- Pérdida: MSE.

## Paso a paso detallado

1. **Construir el dataset**
   - Arma un vector $X = [0.0, 0.1, 0.2, \ldots, 1.0]$. [1]
   - Arma un vector $Y$ con los valores correspondientes de non-linearity en porcentaje. [1]
2. **Preprocesamiento**
   - Escala la presión a $[0, 1]$ (en realidad ya está casi en ese rango).
   - Opcional: escala la no linealidad multiplicándola por 100 si quieres trabajar en unidades más grandes, aunque en % ya es interpretable.

3. **Definir la red neuronal (gemelo digital)**
   - MLP 1D: Input(1) → Dense(20, tanh) → Dense(20, tanh) → Dense(1, lineal).
   - Una activación como tanh es buena para capturar cambios suaves y simétricos.
4. **Entrenamiento con mínimo error**
   - Usa MSE como pérdida y Adam como optimizador.
   - Debido a que solo tienes 11 puntos, puedes usar validación cruzada o simplemente entrenar con todos pero controlando el sobreajuste (pesos pequeños, pocas épocas).
   - Entrena hasta que el error medio sea muy pequeño (por ejemplo, error típico < 0.005% de non-linearity).
5. **Graficar el comportamiento**
   - Gráfica original (puntos): presión vs non-linearity usando los datos de la tabla.[1]
   - Curva de la red: genera una malla densa de presión entre 0 y 1 Bar y calcula el valor predicho por la red.
   - En la misma gráfica, muestra:
     - Puntos reales (marcadores).
     - Curva del gemelo digital (línea continua).
6. **Analizar la curva resultante**
   - Verifica que el gemelo digital reproduce:
     - Pico máximo cerca de 0.7 Bar (~0.12%).
     - Valor casi cero en 0.0 y 1.0 Bar.
     - Cambios de signo entre 0.2–0.3 (pasa por valores negativos) y en 0.9 (valle negativo).[1]
   - Comenta que dichos comportamientos son importantes para corregir la medición del sensor y que tu red puede servir luego para compensar estas desviaciones.
7. **Índice de error mínimo**
   - Calcula MSE, MAE y error máximo absoluto.
   - Dado que la no linealidad está entre –0.05% y 0.12%, errores del orden de 0.001–0.005% ya son muy buenos.[1]
   - Reporta en la respuesta:
     - "MSE en validación = … $\%^2$"
     - "MAE = … %"
     - "Error máximo absoluto = … %".

### Dudas / cosas no claras

- No se especifica claramente si el gemelo digital debe modelar **solo la no linealidad** o el comportamiento completo del sensor (entrada presión → salida lectura corregida).
- No se indica si se espera que el gemelo digital se use luego para **corrección de medida** (es decir, aplicar la no-linealidad al valor de presión) o solo para describir la curva.

**Preguntas para tu profesor**

1. ¿El gemelo digital debe predecir **solo el porcentaje de non-linearity** a partir de la presión, o también debe generar la **señal de salida corregida del sensor**?
2. ¿Qué nivel de precisión considera aceptable para el gemelo digital (por ejemplo, error máximo en % de non-linearity)?
3. ¿Desea que representemos también las **curvas de error** (por ejemplo, error de la red vs presión) además de la curva principal presión–non-linearity?

Si quieres, en un siguiente mensaje puedo bajarlo aún más al nivel de código (por ejemplo, pseudocódigo o ejemplos en Python/Keras) para cada problema, pensando en cómo presentar los gráficos y los índices de error en tu entrega.

⁂

1. EXAMEN-PARCIAL-No1-SISTEMAS-INTELIGENTES.pdf